Selenium-Framework Overview

This project is a full-stack, enterprise-grade automation framework built over 60 days, combining API and UI testing into a clean, modular, and scalable architecture. It includes a domain-driven API client layer, a robust Page Object Model for UI flows, multi-environment test data management, JSON schema validation, AI-assisted scenario generation, and comprehensive utilities for logging, reporting, and artifact handling. The framework supports positive, negative, and contract testing across both layers, integrates seamlessly with CI through GitHub Actions, and produces rich Allure reports for traceability. Designed with onboarding clarity and long-term maintainability in mind, it demonstrates senior-level engineering practices, clean abstractions, and production-ready automation capabilities.

- API Module
  - High-level summary
    - The API module provides a layered, reusable, domain-driven structure for all HTTP-based testing. It includes transport, response modeling, shared abstractions, and endpoint-specific clients.
  - ApiClient
    - Low-level HTTP engine
    - Builds and sends requests
    - Wraps responses into ApiResponse
    - Ensures consistent transport behavior
  - ApiResponse
    - Normalized response wrapper
    - Provides JSON parsing and helpers
    - Supports schema validation
    - Simplifies assertions
  - BaseApi
    - Shared abstraction for all API clients
    - Loads base URL and headers
    - Provides get/post/put/delete helpers
    - Integrates Allure logging and config
  - AuthApi
    - Authentication domain client
    - Login, logout, token refresh
  - UsersApi
    - CRUD operations for users
  - PostsApi
    - CRUD operations for posts
  - CommentsApi
    - CRUD operations for comments

- UI Page Objects Module
  - High-level summary
    - The UI module implements a clean Page Object Model (POM) that encapsulates locators, actions, waits, navigation, and assertions for all UI flows. It isolates Selenium/WebDriver logic from test logic, improving stability, readability, and long-term maintainability.

- BasePage
    - Core POM abstraction
    - Provides WebDriver access and explicit waits
    - Wraps element interactions with safety and logging
    - Centralizes navigation and exception handling
    - Ensures consistent behavior across all page classes
- HomePage
    - Landing page object
    - Provides navigation to feature pages
    - Validates presence of key UI elements
- LoginPage
    - Encapsulates login form interactions
    - Handles username/password entry and form submission
    - Provides validation for error states
- SuccessfulLoginPage
    - Represents authenticated state
    - Validates successful login
    - Provides navigation hooks for post-login flows
- TablePage
    - Encapsulates table interactions
    - Supports sorting, row/column validation, and data extraction
    - Integrates with TableUtils for complex table logic
- DynamicControlsPage
    - Models dynamic UI behavior (add/remove elements, enable/disable fields)
    - Provides wait logic for asynchronous UI changes
    - Validates dynamic state transitions
- DragAndDropPage
    - Encapsulates drag-and-drop interactions
    - Provides stable actions for HTML5 drag-and-drop
    - Validates element repositioning
- FramesPage
    - Handles multi-frame navigation
    - Provides methods to switch between frames safely
- IFramePage
    - Encapsulates interactions inside a single iframe
    - Provides editor/text-area interactions
- NestedFramesPage
    - Models nested frame structures
    - Provides navigation utilities for multi-level frames
- ExceptionsPage
    - Used for testing error-handling and unexpected UI behavior
    - Validates framework resilience to broken or missing elements

---

- Utilities Module
    - High-level summary
        - The utilities module provides shared, cross-cutting functionality used by both API and UI layers. It centralizes data parsing, schema validation, Allure reporting, artifact

management, AI-driven scenario generation, and table utilities. This layer reduces duplication and ensures consistent, reliable behavior across the entire framework.

- CSVUtils
  - Parses CSV test data
  - Validates structure and throws CsvParsingException on errors
  - Supports data-driven testing for login and table scenarios
- JsonUtils
  - Loads and parses JSON files
  - Supports environment-based test data
  - Provides helpers for extracting fields and validating structure
- TestDataManager
  - Centralized test data loader
  - Selects qa/stage/prod data based on config
  - Provides unified access to JSON and CSV data sources
- SchemaValidator
  - Validates API responses against JSON schemas
  - Ensures contract stability across environments
  - Integrates with ApiResponse and Allure logging
- AllureApiLogger
  - Attaches request/response details to Allure reports
  - Provides traceability for debugging API failures
  - Standardizes logging across all API clients
- AllureReportUtil
  - Attaches screenshots, logs, and artifacts to Allure
  - Supports UI and API test reporting
  - Ensures consistent artifact formatting
- ArtifactManager
  - Manages screenshots, logs, and temporary files
  - Ensures artifacts are stored and cleaned consistently
- TableUtils
  - Provides helpers for parsing, validating, and comparing table data
  - Supports sorting, row/column extraction, and negative scenarios
- TableScenario
  - Defines structured table validation scenarios
  - Used by TableTest and NegativeTableTest
- NegativeTableScenario
  - Generates invalid or edge-case table scenarios
  - Supports negative UI testing
- AiDataGenerator
  - Generates dynamic test data using AI-driven logic
  - Supports API payload creation and negative testing
- AiScenarioGenerator
  - Builds complex scenario outlines for UI and API tests
  - Useful for exploratory and negative testing
- NegativeScenarioGenerator
  - Produces invalid payloads and edge cases for API tests
  - Ensures robust negative coverage

- Data Management Module
  - High-level summary
    - The data management module centralizes all test data loading, parsing, and environment selection. It supports JSON, CSV, and multi-environment data sources, ensuring consistent, reliable, and reusable test inputs across both API and UI layers. This module enables clean data-driven testing and eliminates hardcoded values from test logic.
  - TestDataManager
    - Centralized data loader for all tests
    - Selects qa/stage/prod data based on config.json
    - Provides unified access to JSON and CSV files
    - Ensures consistent data retrieval across modules
  - JsonDataProvider
    - Supplies JSON-based test data to TestNG tests
    - Loads structured objects from environment-specific JSON files
    - Supports API payloads, login data, and complex scenarios
  - LoginDataProvider
    - Provides login test data from CSV and JSON sources
    - Validates structure using CSVUtils and JsonUtils
    - Supports positive and negative login scenarios
  - ApiDataProviders
    - Supplies API payloads for CRUD operations
    - Generates valid and invalid request bodies
    - Integrates with NegativeScenarioGenerator for edge cases
  - loginData.json (qa/stage/prod)
    - Environment-specific login credentials
    - Supports multi-environment execution
    - Ensures realistic test coverage across environments
  - loginData.csv
    - CSV-based login dataset
    - Supports bulk login testing and negative scenarios
  - JSON Schemas (schema folder)
    - Defines expected structure for API responses
    - Used by SchemaValidator for contract testing
    - Ensures API stability and backward compatibility

- Base Test Layer
  - High-level summary
    - The base test layer provides the shared setup, teardown, configuration, and lifecycle management for all API and UI tests. It centralizes driver initialization, environment loading, listeners, retries, and reporting. This ensures consistent execution behavior across the entire framework and eliminates duplication in individual test classes.
  - BaseTest (UI)
    - Core parent class for all UI tests
    - Initializes WebDriver via WebDriverFactory
    - Loads configuration from ConfigManager
    - Manages browser lifecycle (setup/teardown)
    - Captures screenshots and artifacts on failure

        - Integrates TestListener and Allure reporting
    - ApiBaseTest (API)
        - Parent class for all API tests
        - Loads base URL and environment config
        - Initializes API clients (UsersApi, PostsApi, etc.)
        - Provides shared schema validation hooks
        - Ensures consistent API test setup
    - AssertionHelper
        - Provides reusable assertion wrappers
        - Adds descriptive failure messages
        - Integrates with Allure for better reporting
        - Ensures consistent assertion style across tests
    - RetryAnalyzer
        - Implements retry logic for flaky tests
        - Retries failed tests based on configured rules
        - Helps stabilize CI execution
    - RetryListener
        - Attaches RetryAnalyzer to TestNG tests
        - Ensures retry logic is applied automatically
    - TestListener
        - Global TestNG listener
        - Captures test start/finish events
        - Attaches logs, screenshots, and artifacts to Allure
        - Handles failure reporting and cleanup

- Test Suites Module
    - High-level summary
        - The test suites module contains all executable API and UI tests. It demonstrates how the framework's abstractions come together: API clients, Page Objects, data providers, schema validation, listeners, retries, and reporting. Each suite is organized by feature and test type, ensuring clarity, scalability, and real-world coverage across positive, negative, and contract tests.
    - API Test Suites
        - ApiTests
            - End-to-end validation of core API flows
            - Uses UsersApi, PostsApi, CommentsApi
            - Ensures CRUD operations behave as expected
        - ContractTests
            - Validates API responses against JSON schemas
            - Ensures backward compatibility and contract stability
        - CreateUsersTests / CreatePostsTests / CreateCommentsTests
            - Positive creation flows
            - Validates payload structure, status codes, and response bodies
        - UpdateUsersTests / UpdatePostsTests / UpdateCommentsTests
            - Validates update operations
            - Ensures partial and full updates behave correctly
        - DeletePostsTests

            - Validates deletion behavior and response codes
            - Ensures resources are removed or marked deleted
        - GetPostsTests
            - Validates retrieval of single and multiple posts
            - Ensures pagination and filtering behavior
        - NegativeApiTests
            - Centralized negative testing suite
            - Uses NegativeScenarioGenerator for invalid payloads
            - Ensures API handles errors gracefully
        - NegativeUsersTests / NegativePostsTests / NegativeCommentsTests
            - Domain-specific negative scenarios
            - Validates error codes, messages, and schema compliance
        - UI Test Suites
            - LoginTest
            - Validates login flow using LoginPage and SuccessfulLoginPage
            - Covers positive and negative login scenarios
        - TableTest
            - Validates table sorting, row/column extraction, and data integrity
            - Integrates with TableUtils and TableScenario
        - NegativeTableTest
            - Uses NegativeTableScenario to validate error handling
            - Ensures UI responds correctly to invalid or unexpected table states
        - DynamicControlsTest
            - Validates dynamic UI behavior (add/remove, enable/disable)
            - Ensures proper waiting and state transitions
        - DragAndDropTest
            - Validates HTML5 drag-and-drop interactions
            - Ensures stable element repositioning
        - FramesTest
            - Validates navigation across frames and nested frames
            - Ensures correct context switching
        - ExceptionsTest
            - Validates framework resilience to broken or missing elements
            - Ensures exception handling and reporting work correctly
        - SchemaValidationTest
            - UI-to-API integration test
            - Validates UI-driven API responses against JSON schemas
            - Demonstrates cross-layer validation

- Configuration & Exceptions Module
    - High-level summary
        - This module provides the foundational configuration, driver management, and error-handling infrastructure for the entire framework. It centralizes environment settings, WebDriver creation, and custom exceptions, ensuring predictable behavior, clean failure reporting, and consistent setup across all API and UI layers.
    - ConfigManager

        - Loads configuration from config.json
        - Provides access to environment, base URLs, browser settings, and timeouts
        - Ensures consistent configuration across API and UI modules
        - Prevents hardcoded values in tests and page objects
    - WebDriverFactory
        - Creates and configures WebDriver instances
        - Supports multiple browsers and headless modes
        - Applies timeouts, window settings, and driver options
        - Ensures consistent browser lifecycle management for BaseTest
    - CsvParsingException
        - Thrown when CSVUtils encounters malformed or invalid CSV data
        - Provides clear error messages for debugging data-driven tests
    - InvalidTestDataException
        - Raised when JSON or CSV test data is missing required fields
        - Ensures test failures are explicit and easy to diagnose
    - ElementNotFoundException
        - Custom exception for missing UI elements
        - Used by BasePage to provide clearer failure messages than Selenium defaults
    - PageNavigationException
        - Thrown when navigation to a page fails or an expected page state is not reached
        - Helps identify broken flows or incorrect page transitions
    - FrameworkInitializationException
        - Raised when core components (config, drivers, environment) fail to initialize
        - Ensures early, explicit failure instead of silent misconfiguration

---

- Project Infrastructure Module
    - High-level summary
        - The project infrastructure module defines how the framework is executed, reported, organized, and integrated into CI/CD. It includes the GitHub Actions workflow, Allure reporting configuration, TestNG suite setup, logging configuration, and the overall project folder structure. This module ensures the framework runs consistently across local and CI environments and produces clean, traceable reports.
    - GitHub Actions CI (ci.yml)
        - Executes full UI + API test suite on push and pull requests
        - Installs dependencies and sets up JDK and Maven
        - Runs TestNG suite and generates Allure results
        - Uploads Allure report as a CI artifact
        - Ensures consistent, automated validation of every commit
    - Allure Configuration (categories.json, environment.properties, executor.json)
        - Defines reporting categories for failures
        - Captures environment metadata (browser, OS, environment)
        - Records CI executor information for traceability
        - Produces clean, navigable Allure reports
    - testng.xml
        - Central test suite configuration
        - Defines test groups, parallel execution, and listeners
        - Ensures consistent test ordering and execution behavior

- logback-test.xml
    - Configures logging for UI and API tests
    - Controls log levels and output formatting
    - Ensures clean, readable logs for debugging
- testData folder (qa/stage/prod)
    - Stores environment-specific JSON test data
    - Supports multi-environment execution
    - Ensures realistic test coverage across environments
- schema folder
    - Stores JSON schemas for contract testing
    - Used by SchemaValidator to validate API responses
    - Ensures API stability and backward compatibility
- allure-results / logs / target folders
    - Store execution artifacts, logs, and compiled output
    - Used by CI and Allure for reporting and debugging
- pom.xml
    - Defines project dependencies (Selenium, TestNG, Allure, JSON, etc.)
    - Configures build plugins and test execution
    - Ensures consistent dependency management
- README.md
    - Provides project overview, setup instructions, and architecture summary
    - Serves as onboarding documentation for new engineers